Independent replication and dual-kernel verification of the ” 2/3 of the zeros of on the critical line” formalization

Riemann Labs (primaryhosting)

2026-08-11

Contents

1 Independent replication and dual-kernel verification of the “$\geq$ 2/3 of the zeros of on the critical line” formalization

Status: DRAFT for internal review (Chris). Not published, not pushed, no PR. **Verifier:** Riemann Labs / primaryhosting (third party). **Not the author of the result. Date of verification run:** 2026-08-11 (CI on GitHub-hosted runners).

1.1 1. What was verified, and by whom

The theorem and its Lean 4 formalization are the work of **Claude (a large language model developed by Anthropic, PBC)**. The paper is *“More than two thirds of the zeros of the Riemann zeta function lie on the critical line”* (author: Claude; dated August 10, 2026; `/Users/acutis/Desktop/zeta-two-thirds.pdf`). Its §1.6 states plainly that “the author of this paper is a large language model developed by Anthropic” and that “a Lean 4 formalisation of Theorems A–E accompanies the paper.” The upstream Lean repository is `github.com/anthropics/zeta-23-lean`.

Our contribution is independent third-party verification of the formalization — not authorship, and not mathematical peer review of the paper. We took the upstream Lean sources, pinned the exact toolchain, re-ran the full build and the enforced axiom audit, and re-ran the `leanprover/comparator` statement-equality check with a **second, independent kernel** (nanoda) enabled. We did this on our own fork, `github.com/primaryhosting/zeta-23-lean`, branch `replication`, under CI we control. What we can attest to is a formal-methods claim: *the Lean proof is sound under a small trusted base, and it states the theorem the paper claims.* Whether the underlying mathematics is correct in the ordinary sense is a separate question that belongs to human peer review, which is pending.

1.2 2. The result (as stated in the paper), and its scope

The headline is a **proportion** result. From the abstract, with `N(T,2T)` the number of zeros `=` `+ i$\gamma$` of with `T < $\gamma$ $\leq$ 2T` counted **with multiplicity**, and `N$_0$*(T,2T)` the number of **distinct** such zeros with `= 1/2`:

> liminf_{T→∞} N$_0$*(T,2T) / N(T,2T) ≥ 2/3, unconditionally.

This improves the long-standing record for the on-line proportion from Levinson's method — most recently > 5/12 = 0.4166… (Pratt–Robles–Zaharescu–Zeindler, standing since 2020, per the paper's §1.2) — to 2/3. The paper also gives ≥ (2/3 − o(1))N simple zeros on the line (Theorem B), ≥ (5/6 − o(1))N distinct zeros (Theorem C), the Montgomery–Taylor optimal-window constants 0.6725… (Theorem D), and Dirichlet-L analogues (Theorem E).

Scope — read this before quoting the number. The paper's §1.5, "What the results are not," is explicit: the results "have no bearing on the Riemann hypothesis in either direction. The argument produces lower bounds only: it certifies that at least two thirds of the zeros are on the line, and says nothing about the remaining third." The 2/3 constant is **multiplicity-aware** in the strong direction: the denominator `N` counts with multiplicity, the numerator `N$_0$*` counts distinct on-line points. In the trusted comparator statements, Theorems B and C appear in two forms — the base configuration uses the weaker **Cauchy–Schwarz** constants (1/2 simple, 3/4 distinct), and a separate `Multiplicity` topic carries the paper's stated 2/3 / 5/6 constants; the headline 2/3 on-line (Theorem A) is identical in both.

1.3 3. The trusted base

`leanprover/comparator` (the Lean FRO's trusted-verification tool) builds a *trusted* challenge module and an *untrusted* solution module in a sandbox, exports both, checks the solution proves **exactly** the challenge statements (every constant they mention must coincide), that the proofs use **only** the axioms `propext`, `Classical.choice`, `Quot.sound`, and replays the solution through the Lean kernel and, optionally, the independent nanoda kernel.

So a skeptical reader has to trust only:

- Mathlib's own definitions — `riemannZeta`, `DirichletCharacter.LFunction`, `analyticOrderAt`, `Set.ncard`, `finsum`;
- two short trusted files — `comparator/ChallengeDeps.lean` (the counting functions, $ $60 lines of definitions from Mathlib alone) and `comparator/Challenge.lean` (the theorem statements, proofs `sorry`);
- the Lean kernel (and here also the independent nanoda kernel);
- comparator's own assumptions.

Nothing in the ~300-file `Zeta23/` proof library needs to be read to know *what* is proved. That is the point of the architecture, and it is why an independent party can verify faithfulness cheaply.

1.4 4. The headline Lean statement and the trusted definitions

From `comparator/Challenge.lean` (trusted; proof is a deliberate `sorry` on the challenge side — comparator supplies the real proof from the solution side and checks statement equality):

```
theorem two_thirds_on_critical_line :
     $\varepsilon$ > 0,   T$_0$ : $\mathbb{R}$,   T $\geq$ T$_0$, (2 / 3 - $\varepsilon$) * (No
  sorry
```

This is the ε-form of `liminf_{T$\to$∞} N$_0$*(T,2T)/N(T,2T) $\geq$ 2/3`. The objects in it are defined in `comparator/ChallengeDeps.lean` **from Mathlib alone** (no import of `Zeta23`):

```
def IsNontrivialZero (  : $\mathbb{C}$) : Prop := riemannZeta   = 0   0 <  .re    .re < 1
def zeroMult (  : $\mathbb{C}$) : $\mathbb{N}$ := (analyticOrderAt riemannZeta  ).toNat
def zerosIn (T  T  : $\mathbb{R}$) : Set $\mathbb{C}$ := {  | IsNontrivialZero     T  <  .im    .im
def Ncount (T  T  : $\mathbb{R}$) : $\mathbb{N}$ := $\sum$    $\in$ zerosIn T  T , zeroMult
def N0star (T  T  : $\mathbb{R}$) : $\mathbb{N}$ := (zerosIn T  T    {  |  .re = 1 / 2}).ncard  --
```

We read these directly. The zeta function is Mathlib's **actual** `riemannZeta`; a "nontrivial zero" is a zero in the open critical strip `0 < Re   < 1`; multiplicity is Mathlib's `analyticOrderAt` (`.toNat` sends $\to 0$, so `1 $\leq$ zeroMult` encodes a genuine finite-order zero). `Ncount` is multiplicity-weighted; `N0star` counts distinct on-line points — the strong direction. The Dirichlet statements (Theorem E) correctly carry the hypotheses `1 < q` and ` .IsPrimitive`. This is a faithful encoding of "$\geq 2/3$ of 's nontrivial zeros (distinct, on the line) among all zeros counted with multiplicity," not a weakened restatement.

1.5 5. The verification chain (with CI evidence)

All of the following is CI run **31458241556** on branch `replication` of `github.com/primaryhosting/zeta-23-lea` (two earlier runs, 31438375668 and 31454943945, were scaffolding). Toolchain pinned to `leanprover/lean4:v4.33.0-rc2`; Mathlib rev `51e6992efd06126df61a496bebf8f49482a4e129`; `lean4export` rev `b18d673`. Every job checked out the pinned Mathlib revision (visible in the logs: `info: mathlib: checking out revision '51e6992…'`).

Job `build-and-audit` (job ID 93676380718):

1. `lake exe cache get` → `lake build`: the full `Zeta23` closure builds — **9010 jobs**, no errors, no `sorry` warnings (log: `[…/9010] Built …`).
2. **Enforced `#print axioms` audit** over the comparator statements plus the   and PairCeiling theorems. Every headline statement prints exactly the three standard axioms, e.g.:

```
'two_thirds_on_critical_line' depends on axioms: [propext, Classical.choice, Quot.sound]
'five_sixths_distinct' depends on axioms: [propext, Classical.choice, Quot.sound]
'dirichlet_montgomery_taylor_distinct' depends on axioms: [propext, Classical.choice, Quot
```

`'xiPrime_simple_zeros_on_critical_line' depends on axioms: [propext, Classical.choice, Quo`

42 statements print exactly `[propext, Classical.choice, Quot.sound]`. Two further numeric PairCeiling lemmas print a strict subset (`'…LawN256_check' depends on axioms: [propext]` and `'…LawN256_edge' does not depend on any axioms`). **No line contains `sorryAx`, and no line contains a project-specific axiom.** The step then asserts this (`--- assert: every line is exactly the 3 standard axioms, none else`) and the job passes, so the audit is *enforced*, not merely printed.

Comparator matrix (three configs, each `enable_nanoda: true`): each job runs comparator in a sandbox — statement-equality against the trusted `Challenge*` files **plus replay through both the Lean default kernel and the independent nanoda kernel**. All three pass, and each log carries the three decisive lines verbatim:

- `comparator (config)` — job ID 93676380657:

  ```
  Nanoda kernel accepts the solution
  Lean default kernel accepts the solution
  Your solution is okay!
  RESULT config: OK via nanoda+Lean (independent second kernel)
  ```

- `comparator (config-multiplicity)` — job ID 93676380651: same three lines (`Nanoda kernel accepts the solution` / `Lean default kernel accepts the solution` / `Your solution is okay!`).

- `comparator (config-xiprime)` — job ID 93676380706: same three lines.

Artifacts (downloadable from the run, `repos/primaryhosting/zeta-23-lean/actions/runs/31458241556`): `axioms-report` (ID 9089301373), `comparator-config` (9089276195), `comparator-config-multiplicity` (9089183186), `comparator-config-xiprime` (9089335471).

Our own review (bounded claim): a hostile multi-lens referee pass over the statement layer found no mathematical gap, and a faithfulness audit confirmed the Lean statements faithfully encode "$\geq 2/3$ (distinct, on the line) vs all zeros with multiplicity" against Mathlib's real `riemannZeta`. We state this only as *no gap found by our review* — it is not a substitute for peer review.

1.6 6. How to reproduce

Locally, from the repository root (with the pinned toolchain in `lean-toolchain`):

```
lake exe cache get                                   # prebuilt Mathlib for the pinned commit (~5-7
lake build                                           # full Zeta23 closure (9010 jobs)
lake build Solution && lake env lean comparator/PrintAxioms.lean
# every line must read: '<name>' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Full external-kernel check (needs `elan`, `landrun`, a matching `lean4export`, the `comparator` binary, and `nanoda_bin`; per-topic configs run independently):

```
lake env /path/to/comparator comparator/config.json              # $\to$ "Your solution is oka
lake env /path/to/comparator comparator/config-multiplicity.json
lake env /path/to/comparator comparator/config-xiprime.json
```

Or simply inspect our CI: `gh run view 31458241556 --repo primaryhosting/zeta-23-lean`, and `gh run view --job=<id> --log` for any job above. Note: `lake build Challenge` will emit `declaration uses 'sorry'` warnings — those are the **deliberate** placeholder proofs in the trusted statement files, and are expected.

1.7 7. What this is, and what it is not

It is: - An independent, reproducible confirmation that the Lean formalization **builds** sorry-free (9010 jobs), depends on **only the three standard Lean axioms** (no `sorryAx`, no project axiom), and passes `leanprover/comparator` statement-equality with replay through **two independent kernels** (Lean + nanoda) on all three configurations. - A faithfulness check that the Lean statements express the paper's claims against **Mathlib's real `riemannZeta` / `DirichletCharacter.LFunction`**, under a trusted base of just Mathlib's definitions, two ~60-line files, and the kernel(s).

It is not: - **Not a proof of the Riemann Hypothesis.** This is a proportion-on-the-line result; the paper's §1.5 says it has no bearing on RH in either direction and certifies a lower bound only. - **Not our theorem.** The result and its Lean proof are Anthropic/Claude's; we replicated the verification. - **Not mathematical peer review.** Formal verification certifies that the machine-checked proof is sound and states the claimed theorem. It does not adjudicate novelty, the modelling choices, or the paper's exposition — human peer review of the paper is pending. (For context on the mathematics: the Montgomery–Taylor constants live in weaker Cauchy–Schwarz forms in the base comparator config; the paper itself, Remark 1.1, notes 2/3 is within 0.016 of the structural ceiling of its own method.)

Draft prepared for Chris Brock's review. Do not distribute, publish, push, or open a PR without his explicit go-ahead. All CI run IDs, job IDs, and artifact IDs above are re-checkable against `primaryhosting/zeta-23-lean` run 31458241556.